

Aim: Implementation of different types of Joins.

Theory:

The SQL Joins clause is used to combine records from two or more tables in a database. A JOIN is a means for combining fields from two tables by using values common to each. The join is actually performed by the 'where' clause which combines specified rows of tables.

Syntax:

SELECT column 1, column 2, column 3...

FROM table_name1, table_name2

WHERE table_name1.column name = table_name2.columnname;

Simple Join:

It is the most common type of join. It retrieves the rows from 2 tables having a common column and is further classified into-

Equi-join :

A join, which is based on equalities, is called equi-join.

Example:

Select * from item, cust where item.id=cust.id;

In the above statement, item-id = cust-id performs the join statement. It retrieves rows from both the tables provided they both have the same id as specified by the where clause.

Since the where clause uses the comparison operator (=) to perform a join, it is said to be equijoin. It combines the matched rows of tables. It can be used as follows:

- ✓ To insert records in the target table.
- ✓ To create tables and insert records in this table.
- ✓ To update records in the target table.
- ✓ To create views.

Non Equi-join:

It specifies the relationship between columns belonging to different tables by making use of relational operators other than '='.

Example:

Select * from item, cust where item.id<cust.id;

Table Aliases

Table aliases are used to make multiple table queries shorted and more readable. We give an alias name to the table in the 'from' clause and use it instead of the name throughout the query.

Self join:

Joining of a table to itself is known as self-join. It joins one row in a table to another. It can compare each row of the table to itself and also with other rows of the same table.

Example:

```
select * from emp x ,emp y where x.salary >= (select avg(salary) from x.emp  
where x. deptno =y.deptno);
```

Inner join:

The INNER JOIN keyword selects all rows from both the tables as long as the condition is satisfied. This keyword will create the result-set by combining all rows from both the tables where the condition satisfies i.e value of the common field will be the same.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
```

```
FROM table1
```

```
INNER JOIN table2
```

```
ON table1.matching_column = table2.matching_column;
```

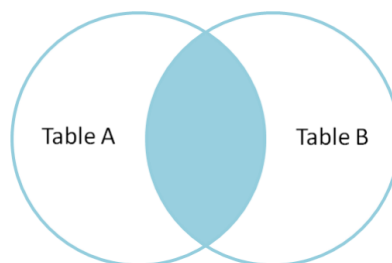
Key Terms

table1: First table.

table2: Second table

matching_column: Column common to both the tables.

Note: We can also write JOIN instead of INNER JOIN. JOIN is same as INNER JOIN.



INNER JOIN Example

Let's look at the example of INNER JOIN clause, and understand it's working. This query will show the names and age of students enrolled in different courses.

Query:

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE FROM Student  
INNER JOIN StudentCourse  
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

Outer Join:

It extends the result of a simple join. An outer join returns all the rows returned by simple join as well as those rows from one table that do not match any row from the table. The symbol(+) represents outer join.

- Left outer join
- Right outer join
- Full outer join

LEFT JOIN

LEFT JOIN returns all the rows of the table on the left side of the join and matches rows for the table on the right side of the join. For the rows for which there is no matching row on the right side, the result-set will contain null. LEFT JOIN is also known as LEFT OUTER JOIN.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
```

```
FROM table1
```

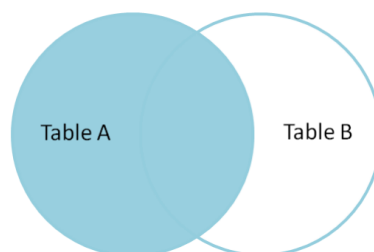
```
LEFT JOIN table2
```

```
ON table1.matching_column = table2.matching_column;
```

Key Terms

- table1: First table.
- table2: Second table
- matching_column: Column common to both the tables.

Note: We can also use LEFT OUTER JOIN instead of LEFT JOIN, both are the same.



LEFT JOIN Example

In this example, the **LEFT JOIN** retrieves all rows from the **Student** table and the matching rows from the **StudentCourse** table based on the **ROLL_NO** column.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
```

```
FROM Student
```

```
LEFT JOIN StudentCourse
```

```
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

RIGHT JOIN

RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. It is very similar to LEFT JOIN for the rows for which

there is no matching row on the left side, the result-set will contain null. RIGHT JOIN is also known as RIGHT OUTER JOIN.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
```

```
FROM table1
```

```
RIGHT JOIN table2
```

```
ON table1.matching_column = table2.matching_column;
```

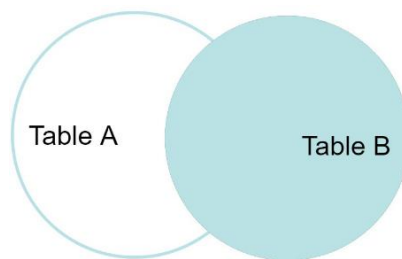
Key Terms

table1: First table.

table2: Second table

matching_column: Column common to both the tables.

Note: We can also use RIGHT OUTER JOIN instead of RIGHT JOIN, both are the same.



RIGHT JOIN Example

In this example, the **RIGHT JOIN** retrieves all rows from the **StudentCourse** table and the matching rows from the **Student** table based on the ROLL_NO column.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
```

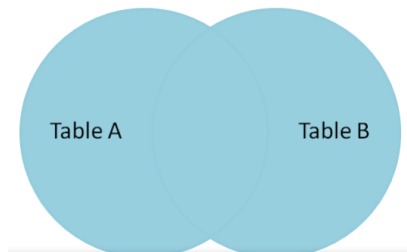
```
FROM Student
```

```
RIGHT JOIN StudentCourse
```

```
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

FULL JOIN

FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.



```
SELECT table1.column1,table1.column2,table2.column1,....
```

FROM table1

FULL JOIN table2

ON table1.matching_column = table2.matching_column;

Key Terms

table1: First table.

table2: Second table

matching_column: Column common to both the tables.

FULL JOIN Example

This example demonstrates the use of a FULL JOIN, which combines the results of both LEFT JOIN and RIGHT JOIN. The query retrieves all rows from the Student and StudentCourse tables. If a record in one table does not have a matching record in the other table, the result set will include that record with NULL values for the missing fields.

Query:

```
SELECT Student.NAME, StudentCourse.COURSE_ID
```

```
FROM Student
```

```
FULL JOIN StudentCourse
```

```
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Natural Join

Natural join can join tables based on the **common columns** in the tables being joined. A natural join returns all rows by matching values in common columns having same name and **data type** of columns and that column should be present in both tables.

- Both table must have at least one common column with same column name and same data type.
- The two table are joined using **Cross join**.
- DBMS will look for a common column with same name and data type. Tuples having exactly same values in common columns are kept in result.

Note: Write a query to be executed in lab with screen shots.

Result: Thus, we execute the different types of joins in SQL successfully.